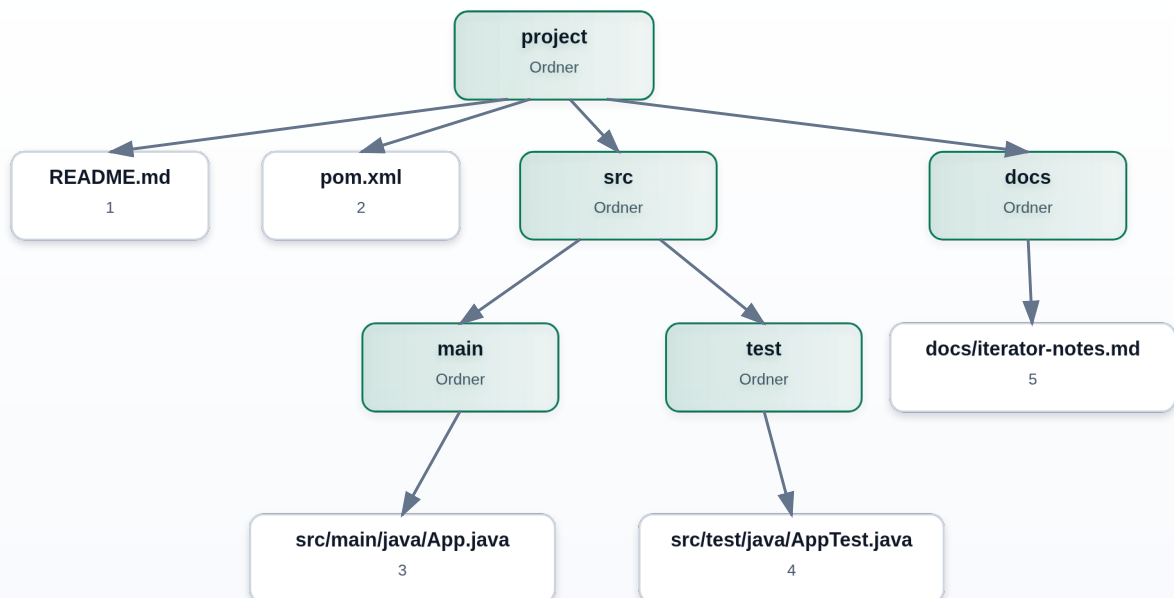


Aufgabe 05: Iterator fuer eine Baumstruktur

Erklaerung mit Ablaufdiagramm, zentralem Iterator-Zustand und Bezug zur konkreten Java-Implementierung.

● Professionelles Ablaufdiagramm



DFS-Reihenfolge: Dateien im Ordner zuerst, danach Unterordner von links nach rechts.

■ Grundidee

Folder bildet eine Baumstruktur:

- Ein Ordner kann Dateien enthalten.
- Ein Ordner kann weitere Unterordner enthalten.
- Diese Unterordner koennen wieder Dateien und weitere Unterordner enthalten.

Der Iterator soll diese verschachtelte Struktur nach aussen wie eine einfache Folge von **FileEntry** - Objekten wirken lassen.

Die gewuenschte Reihenfolge ist Tiefensuche:

```
README.md
pom.xml
src/main/java/App.java
src/test/java/AppTest.java
docs/iterator-notes.md
```

Das bedeutet:

1. Zuerst werden die Dateien des aktuellen Ordners geliefert.
2. Danach wird der erste Unterordner betreten.
3. Dort werden wieder zuerst die Dateien geliefert.
4. Danach folgen dessen Unterordner.
5. Erst wenn dieser Teilbaum fertig ist, geht es mit dem naechsten Ordner weiter.

■ Warum ein Stack?

Ein Iterator muss sich merken, wo er gerade steht. Bei einer einfachen Liste reicht dafuer ein einzelner Index.

Bei einer Baumstruktur reicht ein einzelner Index nicht, weil der Iterator gleichzeitig wissen muss:

- welcher Ordner gerade aktiv ist
- welche Datei in diesem Ordner als naechstes kommt
- welche Unterordner spaeter noch besucht werden muessen

Dafuer verwendet die Loesung einen Stack:

```
private final Deque<FolderState> stack = new ArrayDeque<>();
```

Ein Stack arbeitet nach dem Prinzip: zuletzt hinein, zuerst heraus. Das passt gut zu Tiefensuche, weil immer der zuletzt entdeckte naechste Ordner als erstes weiterbearbeitet wird.

■ Was speichert FolderState?

Der Stack speichert nicht direkt nur `Folder`, sondern `FolderState`:

```
private static final class FolderState {
    private final Folder folder;
    private int fileIndex;
}
```

Ein `FolderState` beschreibt den Zustand fuer genau einen Ordner:

- `folder`: der Ordner selbst

- `fileIndex`: die naechste Datei, die aus diesem Ordner geliefert werden soll

Damit bleibt die Logik uebersichtlich:

- `FolderFileIterator` verwaltet die Reihenfolge der Ordner.
- `FolderState` verwaltet die Position innerhalb eines einzelnen Ordners.

Start im Konstruktor

Der Iterator beginnt beim Wurzelordner:

```
stack.push(new FolderState(Objects.requireNonNull(root)));
```

Damit liegt am Anfang nur der Projektordner auf dem Stack.

Die Rolle von hasNext()

`hasNext()` prueft nicht nur, ob noch etwas vorhanden ist. Die Methode bereitet auch den Stack so vor, dass oben auf dem Stack ein Ordner mit einer noch nicht gelieferten Datei liegt.

```
while (!stack.isEmpty() && !stack.peek().hasNextFile()) {
    FolderState current = stack.pop();
    for (int i = current.folder.folders().size() - 1; i >= 0; i--) {
        stack.push(new FolderState(current.folder.folders().get(i)));
    }
}

return !stack.isEmpty();
```

Solange der Stack nicht leer ist und der oberste Ordner keine Datei mehr liefern kann, wird dieser Ordner vom Stack entfernt.

Danach werden seine Unterordner auf den Stack gelegt.

Warum werden Unterordner rueckwaerts auf den Stack gelegt?

Angenommen ein Ordner hat diese Unterordner:

```
src
docs
```

Die Iteration soll zuerst `src` besuchen und danach `docs`.

Ein Stack liefert aber immer das zuletzt eingefuegte Element zuerst. Deshalb muss `docs` zuerst auf den Stack gelegt werden und danach `src`.

Darum laeuft die Schleife rueckwaerts:

```
for (int i = current.folder.folders().size() - 1; i >= 0; i--) {
    stack.push(new FolderState(current.folder.folders().get(i)));
}
```

Nach dieser Schleife liegt `src` oben auf dem Stack und wird als naechstes bearbeitet.

Die Rolle von next()

`next()` liefert die naechste Datei:

```
if (!hasNext()) {
    throw new NoSuchElementException();
}

return stack.peek().nextFile();
```

Der Aufruf von `hasNext()` ist wichtig. Dadurch wird zuerst sichergestellt, dass oben auf dem Stack wirklich ein Ordner mit einer verfuegbaren Datei liegt.

Wenn keine Datei mehr vorhanden ist, wird wie vom `Iterator`-Vertrag erwartet eine `NoSuchElementException` geworfen.

Wenn eine Datei vorhanden ist, liefert `nextFile()` die aktuelle Datei und erhoeht danach den Index:

```
private FileEntry nextFile() {
    return folder.files().get(fileIndex++);
}
```

Beim naechsten Aufruf ist dadurch automatisch die naechste Datei desselben Ordners an der Reihe.

Zusammenfassung

Die Loesung funktioniert, weil der Iterator zwei Arten von Zustand speichert:

- Den Stack der noch zu bearbeitenden Ordner.
- Den Datei-Index pro aktivem Ordner.

Dadurch kann eine verschachtelte Baumstruktur Schritt fuer Schritt wie eine einfache lineare Folge durchlaufen werden.

Der Client-Code muss nichts ueber die interne Struktur wissen:

```
for (FileEntry file : Folder.sampleProject()) {  
    System.out.println(file);  
}
```

Genau das ist der Kern des Iterator Patterns: Die Traversierung wird gekapselt, und der Aufrufer arbeitet nur mit dem Iterator-Vertrag.